



Universidad Nacional de La Matanza
Departamento de Ingeniería e Investigaciones Tecnológicas

Principios de Diseño de Sistemas (3642)

Curso: 02 4300

Grupo: 9

SEGUNDA EVALUACIÓN

Título: Diseño de un Sistema de Registro de Envíos para TRACEX

Fecha de Presentación: [30/06/2026]

Integrantes

DNI	Nombre	Apellido	Email
45429539	Luciano Nicolas	Barrionuevo	nicolasluciano1425@gmail.com
45233239	Sebastian	Mangalaviti	mangalaviti.sebastian@gmail.com
46364299	Julieta Luciana	Scagni	julietascagni04@gmail.com
46026654	Santiago	Granero	granerosantiago30@gmail.com
41150839	Maximiano	Mascasin Muñoz	maxi.mascasin@gmail.com
44353550	Agostina	Salas	agostina.salas.as@gmail.com

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

1. Consigna.....	3
2. Introducción.....	4
3. Diagrama Casos de Uso.....	5
4. Especificación del Caso de Uso “Solicitar Envío”.....	8
5. Mockup para el Caso de Uso “Solicitar Envío”.....	11
6. Diagrama de Secuencia (Flujo principal).....	14
7. Diagrama de Clases de Diseño.....	16
8. Aplicación de los patrones GRASP.....	20

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

1.Consigna

Una empresa de logística denominada TRACEX desea desarrollar un sistema informático para gestionar de forma integral sus operaciones de envío y distribución de paquetes.

La empresa tiene diferentes tipos de empleados, de los que registra sus datos personales, rol dentro de la empresa y la zona geográfica asignada:

- Operadores de sucursal, encargados de registrar envíos y recepcionar paquetes.
- Repartidores, responsables de la distribución.
- Supervisores, encargados del control operativo.

Cuando un cliente solicita un envío, el operador registra: datos del remitente y destinatario, características del paquete (peso, dimensiones), tipo de servicio (por ejemplo: estándar, urgente, exprés).

El importe del envío se determina automáticamente a partir de distintos factores (como el peso y las dimensiones del paquete, el tipo de servicio contratado —estándar o urgente— y la zona de destino).

Una vez confirmado el pago se genera un código único de seguimiento y se emite la etiqueta del envío.

Durante el ciclo logístico, los paquetes pasan por distintos puntos (clasificación, transporte, distribución).

Cada vez que un paquete cambia de ubicación o estado, el operador correspondiente registra el evento mediante el escaneo del código de seguimiento. Estos registros actualizan automáticamente el estado visible para el cliente en un portal de consulta.

Al llegar a la instancia de entrega:

- Si el destinatario recibe el paquete, el estado se actualiza como entregado.
- Si no se encuentra disponible, el repartidor registra un intento fallido.

En caso de intento fallido, el sistema genera automáticamente una notificación por SMS indicando posibles acciones:

- Primer intento → Reintento de entrega.
- Segundo intento → Retiro en sucursal durante un plazo determinado.

Si se acumulan dos intentos fallidos y no hay respuesta del destinatario dentro del plazo establecido, el envío debe iniciar el proceso de retorno al remitente.

Al finalizar cada jornada, el sistema debe generar un informe que incluya envíos entregados, intentos fallidos, envíos en tránsito, envíos en proceso de devolución.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

2. Introducción

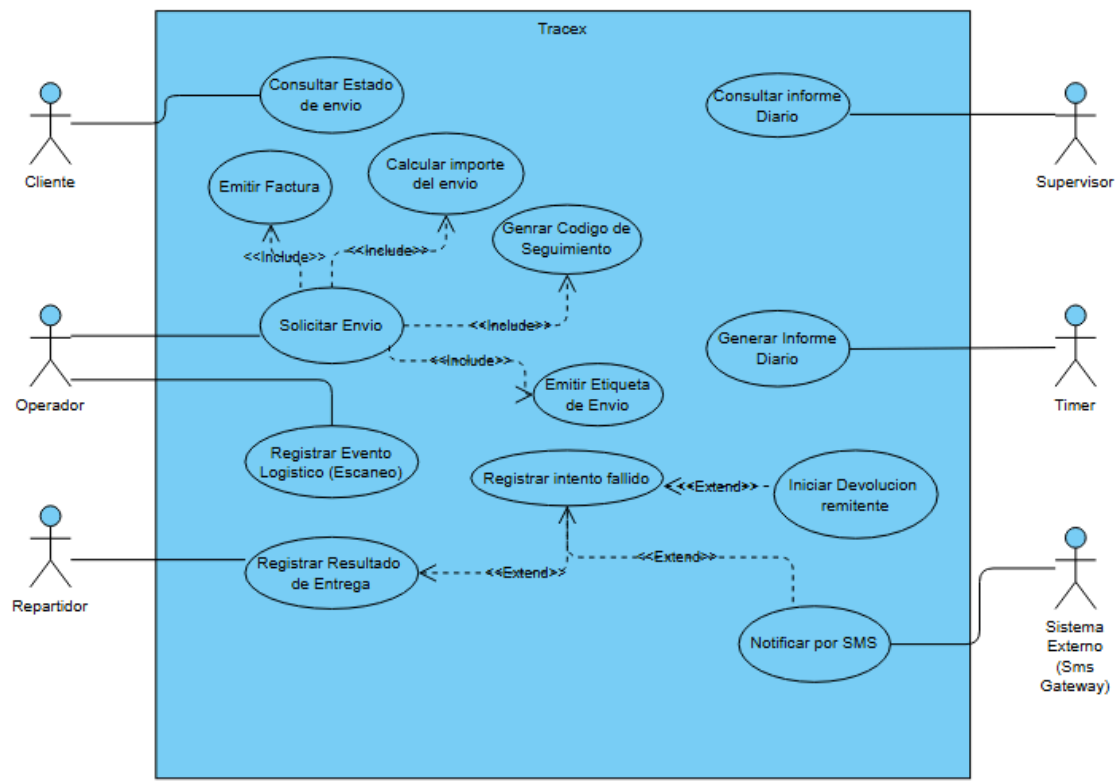
El presente informe describe las principales decisiones de diseño adoptadas durante el desarrollo del sistema **TRACEX**, correspondiente al trabajo práctico final de la asignatura.

Para el modelado del sistema se emplearon diagramas UML de clases de diseño y diagramas de secuencia, buscando representar la estructura del sistema y la interacción entre sus componentes durante la ejecución de los casos de uso principales.

Durante el proceso de diseño se aplicaron distintos patrones GRASP con el objetivo de distribuir adecuadamente las responsabilidades entre las clases, favoreciendo un diseño con alta cohesión, bajo acoplamiento y facilidad de mantenimiento.

Asimismo, se utilizó un asistente basado en inteligencia artificial como herramienta de apoyo para analizar distintas alternativas de modelado, validar decisiones de diseño, verificar la correcta aplicación de los patrones GRASP y explorar mejoras sobre la arquitectura propuesta. Las sugerencias obtenidas fueron evaluadas por el grupo e incorporadas únicamente cuando resultaban consistentes con los requerimientos del sistema y con los principios de diseño orientado a objetos.

3. Diagrama Casos de Uso



Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

Pregunta realizada a la IA	Fragmento de respuesta utilizado	Impacto en el diseño
En el diagrama de casos de uso de TRACEX: el Supervisor no genera el informe, sino que lo consulta; el Repartidor debería tener acciones de registrar entrega/confirmar entrega/registrar resultado con extend; y el Cliente no debería hacer todo lo que hace el Operador. ¿Cómo se corrige esto?	Se identificó que, según el enunciado, el Operador es quien registra la solicitud de envío ("el operador registra..."), por lo que el Cliente solo puede iniciar la consulta de estado del envío. Se propuso que el Supervisor tenga el caso de uso "Consultar Informe Diario" en lugar de generarlo, y que el Repartidor tenga "Registrar Resultado de Entrega" con extend hacia "Confirmar Entrega Exitosa" y "Registrar Intento Fallido".	Redefinió completamente la asignación de actores: el Cliente quedó limitado a "Consultar Estado de Envío"; el Operador absorbió "Solicitar Envío" y "Registrar Evento (Escaneo)"; el Repartidor quedó con "Registrar Resultado de Entrega" y sus extend; el Supervisor pasó de "generar" a "consultar" el informe.
¿El operador registra los datos del cliente como un caso de uso aparte? ¿El cálculo automático del importe lo hace el operador o un actor "Sistema"?	Se explicó que registrar datos del remitente/destinatario es un paso interno del flujo de "Solicitar Envío", no un caso de uso independiente. Y que el cálculo automático del importe se modela como un <<include>> de "Solicitar Envío", sin necesidad de un actor "Sistema", ya que ese rol se reserva para sistemas externos reales (como el SMS Gateway).	Evitó la sobre-fragmentación del diagrama en casos de uso triviales y aclaró el criterio para diferenciar comportamiento interno automático de actores externos genuinos.

Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

¿Generar el informe diario no lo hace nadie? ¿De dónde saldría ese caso de uso si el enunciado dice tanto que el Supervisor genera informes como que el sistema los genera automáticamente al cierre de jornada?	Se planteó resolver la contradicción separando ambos eventos: la generación es automática y disparada por tiempo (cierre de jornada), mientras que la consulta es a demanda del Supervisor. Se sugirió modelar el disparo automático con un actor Timer, una práctica reconocida en UML para representar eventos temporales.	Se incorporó el actor Timer al diagrama, conectado a "Generar Informe Diario" y a "Iniciar Devolución al Remitente" (evento disparado por vencimiento de plazo). Esto agregó complejidad justificada y resolvió la inconsistencia del enunciado con una solución documentada como supuesto.
¿"Consultar Informe Diario" sería un extend de "Generar Informe Diario"?	Se aclaró que <<extend>> es para comportamiento opcional o condicional, y que consultar un informe ya generado no es opcional respecto de su generación, por lo que no corresponde esa relación. Se recomendó modelarlos como dos casos de uso independientes, sin relación entre sí: uno disparado por el Timer y otro iniciado por el Supervisor.	Se eliminó una relación <<extend>> mal aplicada y se dejaron "Generar Informe Diario" y "Consultar Informe Diario" como casos de uso separados, cada uno con su propio actor disparador, lo cual es conceptualmente correcto según la semántica de UML.

Estas interacciones permitieron corregir la asignación inicial de actores (en particular las acciones erróneamente atribuidas al Cliente y al Supervisor), incorporar el actor Timer para modelar eventos automáticos disparados por tiempo, y depurar las relaciones <<include>> y <<extend>> del diagrama según su correcta semántica en UML, evitando la inclusión de casos de uso triviales o relaciones incorrectamente justificadas.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

4. Especificación del Caso de Uso “Solicitar Envío”

Nombre del CU: Solicitar Envío

Tipo de caso de uso: BASE

Objetivo/Descripción del CU: Registrar una solicitud de envío ingresando los datos del remitente, destinatario y paquete, calcular el importe, confirmar el pago, generar un código de seguimiento y emitir la etiqueta para dejar el envío listo para iniciar el proceso logístico.

Actor Principal: Operador

Actor Secundario: Cliente

Fecha creación: 27/06/2026

Precondiciones: Registrar inicio de sesión en el sistema.

Punto de extensión: No

Flujo Normal:

1. El operador inicia la solicitud de envío a petición del cliente.
2. El sistema muestra el formulario de solicitud de envío.
3. El operador completa los datos del remitente y destinatario.
4. El sistema muestra el formulario sobre el paquete.
5. El operador ingresa el peso y dimensión del paquete, y selecciona el tipo de servicio.
6. El sistema calcula el importe en base a estos factores, se incluye el caso de uso “Calcular Importe”
7. El operador recibe el pago de forma externa y registra la confirmación del mismo.
8. El sistema confirma el pago.
9. El sistema genera el código de seguimiento, se incluye el caso de uso “Generar código de seguimiento”.
10. El sistema genera la etiqueta de envío, se incluye el caso de uso “Emitir etiqueta de envío”.
11. El sistema genera la factura, se incluye el caso de uso “Emitir Factura”.
12. El sistema registra el envío.
13. Finaliza el caso de uso.

Postcondición: El envío queda registrado en el sistema, se genera un código único de seguimiento, se emite la etiqueta del envío, el envío queda listo para iniciar el ciclo logístico.

Flujo alternativo:

A1. Datos incompletos o inválidos.

*En el paso 3

3.1 El sistema muestra un mensaje indicando los datos faltantes o incorrectos.

3.2 El operador corrige los datos.

3.3 El caso de uso continúa desde el paso correspondiente.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

3.4 Finaliza el caso de uso

A2. Peso/Dimensiones de paquete inválidos.

*En el paso 6

6.1 El sistema muestra un mensaje indicando que el peso/dimensiones del paquete son incorrectos.

6.2 El operador corrige los datos.

6.3 El caso de uso continúa desde el paso correspondiente.

6.4 Finaliza el caso de uso

A3. Pago no confirmado.

*En el paso 7

2.1 El sistema informa que el pago no pudo confirmarse.

2.2 El operador puede reintentar la operación.

2.3 Si el pago continúa sin confirmarse, el envío no queda registrado y el caso de uso finaliza.

A4. Cancelación de la solicitud

*En cualquier momento antes del paso 7

1. El operador cancela la solicitud.

2. El sistema descarta la información ingresada.

3. Finaliza el caso de uso.

Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

Pregunta realizada a la IA	Fragmento de respuesta utilizado	Impacto en el diseño
¿Qué actividades deben modelarse como casos de uso incluidos (<<include>>)?	Se identificó que Calcular Importe , Generar Código de Seguimiento y Emitir Etiqueta de Envío son pasos obligatorios del proceso.	Se agregaron referencias a los casos de uso incluidos dentro del flujo normal de "Solicitar Envío".
¿Qué flujos alternativos conviene incorporar para cubrir situaciones excepcionales?	Se sugirió contemplar errores de validación, problemas con el pago, error al generar código de seguimiento, y al generar etiqueta de envío.	Se agregaron los flujos alternativos de datos incompletos , peso/dimensiones inválidos , pago no confirmado .
¿Quién debe ser el actor principal del caso de uso "Solicitar Envío": el Cliente o el Operador?	Se concluyó que el Operador es quien interactúa con el sistema para registrar la solicitud, mientras que el Cliente participa proporcionando la información necesaria.	Se definió al Operador como actor principal y al Cliente como actor secundario del caso de uso.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

5. Mockup para el Caso de Uso “Solicitar Envío”

TRACEX - Solicitar Envío
Operador: Juan García

Fecha: 30/06/2026
Hora: 12:00

Datos del Remitente:

Nombre:

DNI:

Teléfono:

Dirección:

Datos del Destinatario:

Nombre:

DNI:

Teléfono:

Dirección:

Datos del Paquete:

Peso (kg):

Dimensiones:

Zona destino:

Tipo de Servicio:

Calcular Importe

Importe:

\$8000,00

Confirmar Pago

Código de seguimiento:

TRX-000001

Registrar envío

Cancelar

Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

Pregunta realizada a la IA	Fragmento de respuesta utilizado	Impacto en el diseño
¿Cómo organizar la pantalla del caso de uso "Solicitar Envío"?	Se propuso dividir la interfaz en secciones: datos del remitente, destinatario, paquete y acciones del envío	Se obtuvo una interfaz ordenada y fácil de comprender.
¿En qué orden debería realizarse el proceso de solicitud de envío?	Se indicó seguir el flujo: completar datos, calcular importe, confirmar pago y registrar el envío.	Se organizó el flujo de la interfaz respetando el proceso definido en el caso de uso.
¿Qué principios heurísticos de Nielsen podrían aplicarse al mockup?	Se propuso aplicar "Visibilidad del estado del sistema" y "Prevención de errores".	Se diseñó una interfaz organizada y con información clara sobre el estado del proceso.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

Principios heurísticos de Jakob Nielsen aplicados al mockup

Visibilidad del estado del sistema

El mockup muestra el importe calculado, la confirmación del pago y el código de seguimiento generado. Este principio permite que el operador sepa en qué etapa del proceso se encuentra. Primero carga los datos, luego calcula el importe, confirma el pago y finalmente visualiza el código de seguimiento. De esta manera, el sistema brinda retroalimentación clara y evita que el usuario dude si la acción fue realizada correctamente.

Prevención de errores

El formulario está dividido en secciones: datos del remitente, datos del destinatario y datos del paquete. Además, se usan listas desplegables para “Zona destino” y “Tipo de servicio”.

Esta organización reduce errores de carga, ya que el operador completa la información de forma ordenada. Las listas desplegables evitan que se ingresen valores inválidos, por ejemplo un tipo de servicio inexistente o una zona mal escrita. Esto mejora la consistencia de los datos y facilita el registro correcto del envío.

6. Diagrama de Secuencia (Flujo principal)



[Link al Diagrama](#)

Pregunta realizada a la IA	Fragmento de respuesta utilizado	Impacto en el diseño
¿En qué punto de la secuencia se debe instanciar el objeto Paquete? ¿Antes de calcular la cotización (para pasar el objeto completo al calculador) o después de que el Operador confirme el cobro físico?	Si instanciamos la entidad persistente Paquete antes de que el cliente apruebe el presupuesto, y el cliente desiste del envío al escuchar el precio, generamos registros innecesarios en la base de datos.	Para cotizar la tarifa antes del cobro, el GestorEnvio llama a CalculadorImporteEnvio.calcularImporte() pasando únicamente parámetros primitivos (peso, dimensiones), garantizando que no se cree ninguna entidad física en memoria hasta que el pago esté garantizado.

Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

Al invocar al GeneradorEtiquetaEnvio, ¿debemos enviarle en el mensaje todos los atributos desagregados del envío (remitente, destinatario, dirección, codigoSeg) o pasarle el objeto Envio completo?	Pasar múltiples parámetros primitivos genera firmas de métodos inestables (si mañana se añade el atributo "prioridad", habría que cambiar el método de la secuencia). Al pasar la instancia completa de Envio, delegamos la lectura al objeto que es experto en sus propios datos.	Se parametrizó el mensaje de creación como generarEtiqueta (nuevoEnvio).
¿Cómo debe modelarse el flujo de confirmación del pago en la secuencia? ¿Debe el EnvioController interactuar de forma separada con el GestorPago y luego con el GestorEnvio, o debe centralizarse la coordinación?	El controlador no debe conocer la coreografía del negocio (saber que tras un pago aprobado se debe generar un código). El controlador solo debe notificar que el usuario disparó una acción. El gestor del caso de uso es quien debe centralizar la orquestación.	El EnvioController envía un único mensaje unificado registrarPagoConfirmado (medioPago) al GestorEnvio. Es el GestorEnvio quien invoca al GestorPago para crear el pago y, ante la respuesta exitosa, dispara las llamadas servicios de generación de código de seguimiento y de impresión de etiquetas.

Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

¿Cómo manejar la generación del código de seguimiento, la etiqueta y la factura?	Las responsabilidades de generación de objetos especializados pueden encapsularse en servicios independientes para mantener alta cohesión.	Se incorporaron los servicios GeneradorCodigoSeguimiento , GeneradorEtiquetaEnvio y GeneradorFactura , evitando sobrecargar a GestorEnvio .
¿Quién debe gestionar el proceso de pago?	El pago constituye una responsabilidad independiente del registro del envío y conviene encapsularlo en un gestor específico.	Se creó GestorPago , encargado de iniciar y confirmar el pago antes de completar el registro del envío.
¿La clase Envío posee demasiadas responsabilidades?	La entidad debe representar el estado del dominio y evitar coordinar el caso de uso. Las responsabilidades de creación, generación y coordinación deben permanecer en los gestores y servicios.	Se simplificó la clase Envio , eliminando métodos de coordinación y dejando únicamente operaciones de consulta como getImporteTotal() , getEstado() y getIntentos() , mejorando la cohesión de la entidad.

Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

¿Es necesaria una clase Scanner, ya que el operador escanea el código de seguimiento?	El enunciado dice 'el operador registra el evento mediante el escaneo' — el sujeto es el operador, no el scanner... Acá hay un humano que hace la acción, entonces el sistema lo recibe como operación del empleado.	Se descartó la clase Scanner. Operador y Repartidor llaman directamente al método registrarEvento() de SeguimientoController, evitando agregar una clase sin justificación textual en el enunciado.
¿Cómo evitar inconsistencias entre el cálculo del importe y la creación del paquete?	El cálculo puede realizarse utilizando los datos ingresados por el usuario sin necesidad de instanciar previamente un objeto Paquete.	Se modificó la firma de calcularImporte() para recibir peso, dimensiones, tipo de servicio y zona , eliminando la dependencia directa con Paquete y manteniendo la consistencia con el diagrama de secuencia.
¿GestorSeguimiento concentra demasiadas responsabilidades?	El gestor debe limitarse a coordinar el registro del seguimiento y delegar las funciones auxiliares a servicios especializados.	Se revisó la distribución de responsabilidades de GestorSeguimiento , manteniendo únicamente la coordinación del seguimiento y delegando las notificaciones al servicio ServicioNotificacionSMS .

Principios de Diseño de Sistemas		TP 01
----------------------------------	--	-------

¿Conviene modelar Informe como clase persistente o solo como un método generarInforme() sin clase asociada?	El enunciado dice 'el sistema debe generar un informe que incluya envíos entregados, intentos fallidos...' Son datos estructurados que necesitan una clase que los represente. Conviene aplicar Creator: GeneradorInforme crea InformeDiario.	Se incorporó la clase InformeDiario como entidad con atributos, generada por GeneradorInforme y consultada por Supervisor.
Para la clase Empleado y sus subclases (Operador, Repartidor, Supervisor), ¿la lógica de registrar envíos, entregas o consultar informes va en las subclases o en los Controllers/Gestores?	Si dos objetos del mismo tipo tienen comportamientos distintos, conviene modelarlos como subclases para respetar High Cohesion... pero si esas acciones son eventos del sistema, esa responsabilidad le corresponde al Controller, no a la entidad que dispara el evento.	Se definió la herencia Empleado → Operador / Repartidor / Supervisor solo con atributos diferenciados, sin métodos propios. La lógica de coordinación se delegó a EnvioController, SeguimientoController y los Gestores, mientras que Envío conservó únicamente métodos de consulta sobre sus propios datos (getImporteTotal, getEstado, getIntentos).
¿Conviene que Pago tenga una clase EstadoPago separada en lugar de un atributo?	Modelar el estado como una clase aparte (EstadoPago) en lugar de un String suelto permite que el dato sea consistente y reutilizable, y deja más clara la relación entre Pago y su estado dentro del diagrama.	Se incorporó la clase EstadoPago como entidad asociada a Pago mediante la relación 'estado', en lugar de modelarlo como un atributo simple dentro de Pago.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

8. Aplicación de los patrones GRASP

Durante el diseño del sistema se aplicaron distintos patrones GRASP (General Responsibility Assignment Software Patterns) con el objetivo de lograr una correcta distribución de responsabilidades entre las clases, favoreciendo un diseño con alta cohesión, bajo acoplamiento y facilidad de mantenimiento. A continuación, se describen los principales patrones utilizados y su aplicación dentro del modelo.

Controller

El patrón **Controller** se aplicó asignando la responsabilidad de recibir y coordinar las solicitudes provenientes de la interfaz de usuario a las clases **EnvioController** y **SeguimientoController**. Estas clases representan el primer punto de contacto entre el usuario y la lógica del sistema, evitando que las entidades del dominio conozcan detalles de la interacción con la interfaz.

En particular, **EnvioController** recibe la solicitud de creación de un nuevo envío y delega completamente su procesamiento a **GestorEnvio**, mientras que **SeguimientoController** canaliza las operaciones relacionadas con el registro de eventos y escaneos hacia **GestorSeguimiento**.

Esta decisión permitió separar la lógica de presentación de la lógica del negocio, evitando que las entidades del dominio asumieran responsabilidades ajenas a su naturaleza y favoreciendo un menor acoplamiento entre las distintas capas del sistema.

Creator

El patrón **Creator** se aplicó principalmente en la clase **GestorEnvio**, responsable de crear la entidad **Envio**.

Esta decisión se fundamenta en que **GestorEnvio** posee toda la información necesaria para inicializar correctamente un envío, incluyendo los datos del remitente, destinatario, paquete, tipo de servicio, zona de destino y el importe calculado. Asimismo, coordina la generación del código de seguimiento, la etiqueta y la factura, por lo que resulta el candidato natural para instanciar el objeto.

La aplicación de este patrón permitió centralizar la creación del envío en una única clase, manteniendo la consistencia del modelo y evitando distribuir dicha responsabilidad entre múltiples objetos.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

Information Expert

El patrón **Information Expert** se utilizó asignando las responsabilidades a las clases que poseen la información necesaria para llevarlas a cabo.

Por ejemplo, la entidad **Envio** conserva la información correspondiente al importe total, el estado actual y los intentos de entrega registrados, por lo que incorpora operaciones de consulta como **getImporteTotal()**, **getEstado()** y **getIntentos()**.

Asimismo, **CalculadorImporteEnvio** es el experto responsable de calcular el costo del envío utilizando el peso, las dimensiones, el tipo de servicio y la zona de destino, evitando que dicha lógica quede distribuida en otras clases del sistema.

Low Coupling

El patrón **Low Coupling** se aplicó buscando minimizar las dependencias entre los distintos componentes del sistema.

Para ello, se separaron responsabilidades específicas en clases especializadas, como **GestorPago**, **CalculadorImporteEnvio**, **GeneradorCodigoSeguimiento**, **GeneradorEtiquetaEnvio**, **GeneradorFactura** y **ServicioNotificacionSMS**.

Esta organización evita que una única clase concentre múltiples responsabilidades y permite modificar o reemplazar cada componente sin afectar significativamente al resto del sistema, favoreciendo su mantenibilidad y escalabilidad.

High Cohesion

El patrón **High Cohesion** se aplicó procurando que cada clase posea una única responsabilidad claramente definida.

Los **Controllers** se limitan a recibir solicitudes del usuario y delegarlas; los **Gestores** coordinan la ejecución de los casos de uso; las **Entidades** representan la información propia del dominio; y los **Servicios** encapsulan funciones específicas como el cálculo del importe, la generación de documentos o el envío de notificaciones.

Principios de Diseño de Sistemas		TP 01
-------------------------------------	--	-------

Durante el proceso de diseño se revisó especialmente la clase **Envio**, reduciendo sus responsabilidades y eliminando operaciones de coordinación que fueron trasladadas a los gestores y servicios correspondientes. Como resultado, la entidad quedó enfocada únicamente en representar el estado del envío y ofrecer las consultas necesarias sobre su información.

Pure Fabrication

Con el objetivo de mantener un bajo acoplamiento y una alta cohesión, se incorporaron clases de servicio que no representan entidades del dominio, sino responsabilidades técnicas específicas.

Entre ellas se encuentran **CalculadorImporteEnvio**, **GeneradorCodigoSeguimiento**, **GeneradorEtiquetaEnvio**, **GeneradorFactura** y **ServicioNotificacionSMS**.

Estas clases encapsulan funcionalidades especializadas que, de incorporarse dentro de entidades como **Envio** o de los gestores, provocarían un aumento innecesario de responsabilidades. La utilización de servicios especializados permitió mantener las entidades del dominio enfocadas en representar la información del negocio, mientras que las tareas auxiliares quedaron centralizadas en componentes reutilizables e independientes.